Q1 **CO4 - AT2 - CASE STUDY** 25 marks

1. An international airport tracks every checked-in bag from check-in counter to aircraft loading. Each bag record must store a bag tag number, passenger information, flight details, and current handling status. The passenger and flight details themselves consist of several related sub-fields that belong together logically. Thousands of bags are processed every hour across multiple terminals, and the system must quickly locate and update any bag's status. The design must remain easy to extend as new tracking checkpoints are added.
Question: Identify the appropriate nested-structure model for a bag record, and explain how a structure pointer can be used by the scanning system to update a bag's status at each checkpoint without copying the entire record.

Your Code

```c
#include <stdio.h>
#include <string.h>
typedef struct
{
    char name[50];
    long passengerId;
} PassengerInfo;

typedef struct
{
    char flightNo[20];
    char origin[30];
    char destination[30];
} FlightDetails;
typedef struct
{
    long bagTagNumber;
    PassengerInfo passenger;
    FlightDetails flight;
    char handlingStatus[30];
} BagRecord;
void updateStatus(BagRecord *ptr, const char *newStatus);
int main()
{
    BagRecord bags[2] =
    {
        {101, {"Jaideep", 5551}, {"AI-101", "Chennai", "Delhi"}, "Check
        {102, {"Priya",  5552}, {"AI-202", "Delhi", "Mumbai"}, "At Secu
    };
    long searchTag = 102;
    char newStatus[] = "Loaded on Aircraft";
    BagRecord *bagPtr = NULL;
    for (int i = 0; i < 2; i++) {
```

Input

```
stdin input
```

Output

```
Bag Tag: 102
New Status: Loaded on Aircraft
```

Q2 **CO4 - AT2 - CASE STUDY** 25 marks

2.A digital wallet application allows users to hold balances and transact in different currency formats, such as standard currency amounts, cryptocurrency amounts, or reward-point balances, but only one format is relevant for a given transaction at a time. Every transaction still carries common information such as transaction ID, timestamp, and user ID regardless of currency type. The application processes a large number of transactions per second and must keep memory usage minimal for each transaction record. The design must clearly justify why a shared memory region is appropriate here.

Question: Design a structure containing a union for the currency-specific amount, and explain why using a union significantly reduces the memory required per transaction compared to storing all currency types separately.

Your Code

```c
#include <stdio.h>
typedef enum
{
    STANDARD,
    CRYPTO,
    REWARD
} CurrencyType;
union CurrencyAmount
{
    double standardAmount;
    float  cryptoAmount;
    int    rewardPoints;
};
struct Transaction
{
    int transactionID;
    long timestamp;
    int userID;
    CurrencyType type;
    union CurrencyAmount amount;
};
int main()
{
    struct Transaction t1 = {101, 1000000, 501, STANDARD, {.standardAm
    struct Transaction t2 = {102, 1000100, 502, CRYPTO,   {.cryptoAmou
    struct Transaction t3 = {103, 1000200, 503, REWARD,   {.rewardPoint
    struct Transaction arr[3] = {t1, t2, t3};
    for (int i = 0; i < 3; i++) {
        printf("Transaction ID: %d, User ID: %d\n", arr[i].transactionI
        if (arr[i].type == STANDARD)
            printf("Type: STANDARD, Amount: %.2f\n", arr[i].amount.star
        else if (arr[i].type == CRYPTO)
```

Input

```
stdin input
```

Output

```
Transaction ID: 101, User ID:
501
Type: STANDARD, Amount: 250.75
----
Transaction ID: 102, User ID:
502
```

Q3 CO4 - AT2 - CASE STUDY

25 marks

3.A city's traffic-signal controller repeatedly executes a signal-change function every time a junction cycle completes, and the transportation department wants to know the total number of signal cycles completed since the controller started running, even though the function itself finishes execution after every cycle. The counter value must persist correctly across thousands of repeated calls without being reset each time. Ordinary local variables would not retain their value between calls, which does not meet the requirement. The department wants a lightweight solution that does not depend on global variables.

Question: Identify which storage class should be used for the cycle counter inside the function, and explain why it is more appropriate than an ordinary local variable for this requirement.

Your Code

```c
#include <stdio.h>
void signalChange()
{
    static int count = 0;
    count++;
    printf("Signal cycle: %d\n", count);
}
int main()
{
    signalChange();
    signalChange();
    signalChange();
    signalChange();
    signalChange();

    return 0;
}
```

Run Save

Input

```
stdin input
```

Output

```
Signal cycle: 1
Signal cycle: 2
Signal cycle: 3
Signal cycle: 4
Signal cycle: 5
```

Q4 CO4 - AT2 - CASE STUDY

25 marks

4. A large enterprise resource planning application is built from multiple source files, including modules for sales, inventory, and finance, and all of these modules must reference a common variable that stores the current fiscal year. The variable is defined once in one source file but must be read and, in some cases, modified by functions compiled separately in other source files. Without proper declaration, each module would either fail to compile or maintain its own inconsistent copy of the fiscal year. The design must ensure a single, consistent value is shared across the entire application.
Question: Identify which storage class allows the fiscal-year variable to be accessed across multiple source files, and explain the role of extern in achieving this shared access.

Your Code

```c
#include <stdio.h>
int fiscal_year = 2026;

void sales()
{
    extern int fiscal_year;
    printf("Sales module: %d\n", fiscal_year);
}
void inventory()
{
    extern int fiscal_year;
    printf("Inventory module: %d\n", fiscal_year);
}
int main()
{
    sales();
    inventory();
    return 0;
}
```

Run Save

Input

stdin input

Output

```
Sales module: 2026
Inventory module: 2026
```